

IP Core Generator (wersja 3)

Treść zadania (przystosowana do laboratorium zdalnego)

Należy wykonać generator epitrochoidy o następujących właściwościach:

1. Figury rysujemy w kwadratowym obszarze o wymiarach 384 x 384 pikseli.
2. Obraz generujemy na analogowym monitorze VGA (możemy wykorzystać tryb 640x480).
3. Położenie kwadratowego obszaru ustawić tak, aby był on możliwie najlepiej widoczny na monitorze VGA przesłoniętym płytką FPGA.
4. Generujemy obraz czarno-biały (wszystkie bity sygnałów RGB sterujemy równocześnie z tego samego źródła). Figura ma być biała, tło czarne.
5. Częstotliwość obu kanałów (f_1 i f_2) ustawiamy za pomocą przełączników SW0-SW7 (zakres 0-255) i przycisków BTN0 (wpis częstotliwości f_1 ustawionej na przełącznikach), BTN1 (wpis częstotliwości f_2 ustawionej na przełącznikach).
6. Amplitudy obu kanałów (A i B) ustawiamy za pomocą przełączników SW0-SW7: (SW7-SW4 – kanał A, 16 pikseli / jednostkę czyli max. 240 pikseli; SW3-SW0 – kanał B, 4 piksele / jednostkę czyli max. 60 pikseli) i przycisku BTN2 (wpis amplitud ustawionych na przełącznikach).
7. Przycisk BTN3 powinien służyć do wyczyszczenia ekranu i zresetowania generatora DDS (aby rozpoczął rysowanie figury od zerowych faz).
8. Współrzędne (0,0) są w środku kwadratowego obszaru.
9. W tym ćwiczeniu nie korzystamy z sygnału „reset”. Proszę zamiast tego stosować wartości początkowe sygnałów i zmiennych.

Informacje o epitrochoidzie:

<https://pl.wikipedia.org/wiki/Epitrochoida>

Wyjaśnienia i wskazówki:

Dla ułatwienia proponowana jest realizacja przekształconych równań epitrochoidy:

$$x = A \cdot \cos 2\pi f_1 t - B \cdot \cos 2\pi f_2 t$$

$$y = A \cdot \sin 2\pi f_1 t - B \cdot \sin 2\pi f_2 t$$

gdzie:

$$A = R + r$$

$$B = h$$

$$f_2 / f_1 = (R + r) / r$$

A, B – amplitudy kanałów

f_1 , f_2 – częstotliwości kanałów

Dzięki możliwości ustawienia częstotliwości f_1 możliwe jest ustawienie niecałkowitych wartości $(R + r) / r$.

W obu kanałach potrzebujemy zarówno wyjścia sinus jak i cosinus.

Przy bardzo dużych nastawach amplitud wykres może wyjść poza obszar wyświetlania powodując zniekształcenia – nie trzeba przed tym zabezpieczać układu.

Wejścia i wyjścia układu:

clk_i – zegar 100MHz,

red_o – wyjście sygnału koloru czerwonego do monitora,

green_o – wyjście sygnału koloru zielonego do monitora,

blue_o – wyjście sygnału koloru niebieskiego do monitora,

hsync_o – wyjście sygnału synchronizacji poziomej do monitora,

vsync_o – wyjście sygnału synchronizacji pionowej do monitora,

sw_i – przełączniki do ustawiania częstotliwości i amplitud kanałów,

btn_i – przyciski do wprowadzania częstotliwości i amplitud kanałów ustawionych na przełącznikach.

Plik z ograniczeniami projektowymi: [isp8a.xdc](#)

Informacje o analogowym sygnale VGA (częstotliwość pikseli można ustalić na równe 25 MHz): [Generowanie sygnału VGA](#)

Parametry sygnału VGA podane w wygodnej formie (w pikselach i liniach):

[Parametry sygnału VGA](#)

Przy generacji sygnału wideo należy pamiętać o zerowaniu sygnałów RGB poza obszarem wyświetlania obrazu (na podstawie poziomu sygnału analogowego poza obszarem wyświetlania monitor ustala poziom czerni dla każdego z kolorów).

Do realizacji projektu potrzebne będą dwa moduły funkcjonalne wygenerowane przez syntezer IP Core. **Dwuportowa pamięć RAM** posłuży jako pamięć obrazu, zaś **Generator DDS (Direct Digital Synthesizer)** wykorzystany zostanie do wytwarzania cyfrowych sygnałów sinus i cosinus.

Generacja modułu dwuportowej pamięci RAM za pomocą syntezy IP core w Vivado

- W okienku **Flow navigator** rozwijamy **PROJECT MANAGER** i klikamy w **IP Catalog**.
- Następnie rozwijamy **Memories & Storage Elements** i dalej rozwijamy **RAMs & ROMs & BRAM**.
- Klikamy podwójnie w **Block Memory Generator**.
- W polu **Component Name** wpisujemy nazwę, którą otrzyma generowany przez nas moduł, np.: **video_mem**
- W zakładce **Basic** zmieniamy **Memory Type** na **Simple Dual Port RAM**. Pamięć dwuportowa będzie nam potrzebna, aby można było równocześnie pisać i czytać z dwóch różnych adresów (zapisujemy piksele wg kształtu krzywej, odczytujemy piksele sekwencyjnie do monitora). Zaznaczamy także opcję **Common Clock** (porty zapisu i odczytu używają tego samego zegara).
- W zakładce **Port A Options** zmieniamy **Port A Width** na **1** (będziemy zapisywać zawsze jeden piksel – bit na określonej pozycji – adresie), następnie zmieniamy **Port A Depth** na **147456** (jest $384 \times 384 = 147456$ pikseli), następnie w polu **Enable Port Type** wybieramy **Always Enabled**.
- W zakładce **Port B Options** zmieniamy **Port B Width** na **1** (będziemy odczytywać zawsze jeden piksel – bit na określonej pozycji – adresie) – możliwe jest ustawienie innej szerokości bitowej magistrali wyjściowej i wejściowej. Liczba adresów jest tym razem wyliczona automatycznie. Następnie w polu **Enable Port Type** wybieramy **Always Enabled** oraz wyłączamy opcję **Primitives Output Register**.
- Znajdujący się w lewym górnym rogu przycisk **Documentation** otworzy szczegółową dokumentację generowanego komponentu.
- W zakładce **Summary** możemy znaleźć podsumowanie istotnych parametrów tworzonego komponentu. Szczególnie istotna jest informacja: **Total Port B Read Latency (From Rising Edge of Read Clock): 1 Clock Cycle(s)**
Oznacza ona, że odczyt z pamięci odbywa się z opóźnieniem 1 cyklu zegara w stosunku do zbocza zegarowego zatrzymującego odczytywany adres (w przeciwieństwie do zapisu, w którym zapisywana dana zatrzymywana jest równocześnie z adresem). W przypadku wątpliwości proszę zajrzeć do dokumentacji komponentu.

- Na zakończenie potwierdzamy ustawienia przyciskiem **OK** w prawym dolnym rogu i po chwili zatwierdzamy w okienku **Generate Output Products** przyciskiem **Generate** rozpoczęcie syntezy nowego komponentu.
- Jeszcze raz potwierdzamy w okienku **Generate Output Products** przyciskiem **OK**.
- Po zakończeniu generacji komponent zostanie dołączony do listy plików źródłowych projektu.

Wygenerowana pamięć posiada dwa wejścia zegarowe (po jednym dla każdego portu) jednak, ponieważ wybraliśmy wcześniej opcję **Common Clock** to musimy oba wejścia zegarowe podłączyć do wspólnego zegara.

Należy zwrócić uwagę na definicję niektórych sygnałów (np. wea, dina, doutb) które są wektorami jednobitowymi – trzeba o tym pamiętać, ponieważ jednobitowy wektor to nie to samo co pojedynczy sygnał.

W celu ułatwienia dołączenia komponentu do hierarchii aktualnego projektu w okienku **Sources** można wybrać zakładkę **IP sources**. Wtedy wyświetlą się wygenerowane moduły IP. Po rozwinięciu wybranego modułu oraz rozwinięciu **Instantiation Template** można zobaczyć plik ***.vho** zawierający szkielet deklaracji komponentu oraz przykład jego użycia (tworzenia instancji). Możemy te elementy łatwo przekopiować do naszego pliku projektowego, w którym planujemy podłączyć wygenerowany komponent, a zaprojektowany komponent zajmie właściwe miejsce w hierarchii projektu.

Istotny fragment przykładowego pliku VHO:

```
-- The following code must appear in the VHDL architecture header.
----- Begin Cut here for COMPONENT Declaration ----- COMP_TAG
COMPONENT video_mem
  PORT (
    clka : IN STD_LOGIC;
    wea : IN STD_LOGIC_VECTOR(0 DOWNTO 0);
    addra : IN STD_LOGIC_VECTOR(17 DOWNTO 0);
    dina : IN STD_LOGIC_VECTOR(0 DOWNTO 0);
    clkb : IN STD_LOGIC;
    addrb : IN STD_LOGIC_VECTOR(17 DOWNTO 0);
    doutb : OUT STD_LOGIC_VECTOR(0 DOWNTO 0)
  );
END COMPONENT;
-- COMP_TAG_END ----- End COMPONENT Declaration -----

-- The following code must appear in the VHDL architecture
-- body. Substitute your own instance name and net names.

----- Begin Cut here for INSTANTIATION Template ----- INST_TAG
your_instance_name : video_mem
  PORT MAP (
    clka => clka,
    wea => wea,
    addra => addra,
    dina => dina,
    clkb => clkb,
    addrb => addrb,
    doutb => doutb
  );
```

```
);
-- INST_TAG_END ----- End INSTANTIATION Template -----

-- You must compile the wrapper file video_mem.vhd when simulating
-- the core, video_mem. When compiling the wrapper file, be sure to
-- reference the VHDL simulation library.
```

Tekst zaznaczony na **czzerwono** należy skopiować do obszaru definicji sygnałów lokalnych. Tekst zaznaczony na **niebiesko** (po ustaleniu nazwy instancji komponentu oraz sygnałów podłączonych) należy skopiować do obszaru opisu układu. Po dokonaniu powyższych modyfikacji ikona komponentu powinna zająć właściwe miejsce w hierarchii plików projektu.

Generacja modułu generatora DDS

- W okienku **Flow navigator** rozwijamy **PROJECT MANAGER** i klikamy w **IP Catalog**.
- Następnie rozwijamy **Digital Signal Processing** i dalej rozwijamy **Waveform Synthesis**.
- Klikamy podwójnie w **DDS Compiler**.
- W polu **Component Name** wpisujemy nazwę, którą otrzyma generowany przez nas moduł, np.: **singen**
- W zakładce **Configuration** ustawiamy **Configuration Options** na **Phase Generator and SIN COS LUT** oraz zmieniamy **Number of Channels** na **2** (będziemy potrzebować dwóch niezależnych generatorów funkcji sinus/cosinus). Konieczne jest także określenie jakości wytwarzanego sygnału (różnica pomiędzy amplitudą prążka pożądanego, a amplitudą największego spośród prążków niepożądanych w widmie sygnału, podawana w dB) – od tego zależy liczba bitów użyta do opisu funkcji sinus/cosinus, a więc także złożoność bloku generatora. Wybieramy – **Spurious Free Dynamic Range (dB): 65**
 Sprawdzamy czy w polu **System Clock (MHz)** jest wpisana właściwa wartość: **100**. Oznacza to, że częstotliwość próbkowania przypadająca na kanał (**Frequency per Channel (Fs)**) wynosi 50 MHz.
 Pozostaje określenie rozdzielczości z którą będziemy ustawiać częstotliwość sygnału – **Frequency Resolution (Hz): 1000**
 Należy także ustawić opcję **Noise Shaping** na **None** (opcja ta służy do modyfikacji właściwości generatora w kwestii jakości generowanego sygnału).
- W zakładce **Implementation** zmieniamy **Phase Increment Programmability** na **Programmable**. Pozwoli to na zmianę częstotliwości podczas pracy generatorów. Częstotliwość będziemy podawać za pomocą przełączników, których stan powinniśmy

wprowadzać bezpośrednio do **najmłodszych** bitów wejścia programującego. Następnie zmieniamy **Phase Offset Programmability** na **None** (w tym układzie nie potrzebujemy różnych faz kanałów). Następnie zmieniamy **Output Selection** na **Sine and Cosine** (w każdym kanale potrzebujemy funkcji sinus i cosinus). Wyłączamy opcję **Has Phase Out** (nie potrzebujemy dodatkowego wyjścia informującego o fazie).

- W zakładce **Detailed Implementation** zaznaczamy (w polu **Control Signals**) opcje: **ACLKEN** oraz **ARESETn**. Wejście aclkenn pozwoli na zatrzymywanie generatora jeżeli nie będziemy gotowi do odbioru danych (normalnie pracuje on cały czas z częstotliwością próbkowania F_s). Wejście aresetn pozwala na uruchomienie obu generatorów równocześnie z nowymi nastawami częstotliwości i zerową fazą początkową, po ich ustawieniu za pomocą przycisków i wyczyszczeniu ekranu przyciskiem **BTN3** – jest to reset synchroniczny aktywny poziomem **niskim**.
- W zakładce **Output Frequencies** ustawiamy częstotliwości początkowe w pierwszym i drugim kanale na: **0.1953125** (MHz)
- Znajdujący się w lewym górnym rogu przycisk **Documentation** otworzy szczegółową dokumentację generowanego komponentu.
- W zakładkach **Summary** i **Additional Summary** możemy znaleźć podsumowanie istotnych parametrów tworzonego komponentu.
- Na zakończenie potwierdzamy ustawienia przyciskiem **OK** w prawym dolnym rogu i po chwili zatwierdzamy w okienku **Generate Output Products** przyciskiem **Generate** rozpoczęcie syntezy nowego komponentu.
- Jeszcze raz potwierdzamy w okienku **Generate Output Products** przyciskiem **OK**.
- Po zakończeniu generacji komponent zostanie dołączony do listy plików źródłowych projektu.

Warto zapamiętać pomocnicze informacje zawierające podsumowanie właściwości projektowanego komponentu (pola **Summary** i **Additional Summary**) oraz blokowy schemat połączeń.

Podobnie jak w poprzednim przypadku należy teraz podłączyć wygenerowany komponent do projektu (można wykorzystać wygenerowany plik *.vho).

Weryfikacja układu

Należy zweryfikować układ praktycznie poprzez zaprogramowanie płytki testowej z układem FPGA. W razie potrzeby można wykonać symulację funkcjonalną (nie jest obowiązkowa). Weryfikacja praktyczna **na zaliczenie** polega na sprawdzeniu działania układu na płycie FPGA poprzez wyświetlenie kilku epitrochoid dla różnych parametrów (i nagranie tego na filmie):

1. Zaraz po zaprogramowaniu płytki powinien się wyświetlić okrąg o średnicy zależnej od ustawień początkowych amplitud A i B (lub kropka jeżeli amplitudy początkowe są zerowe lub takie same – w takim przypadku proszę ustawić przełącznikami amplitudy tak, aby wyświetlił się okrąg).
2. Należy wyświetlić figury dla częstotliwości f_1 ustawionej na 1, a częstotliwości f_2 ustawionej na ostatnią (najmłodszą) cyfrę numeru indeksu studenta zwiększoną o 2. Ustawić kolejno amplitudy A i B na takie wartości aby pokazać epicykloidę (przypadek specjalny epitrochoidy), a następnie epicykloidę skróconą i epicykloidę wydłużoną.
3. Wyświetlić figurę róży: częstotliwość f_1 ustawić na 1, częstotliwość f_2 ustawić na 7, amplitudę A ustawić na 3, amplitudę B ustawić na 12.
4. Kolejna ciekawa figura: nie zmieniając ustawień amplitud ustawić częstotliwość f_1 na 3, częstotliwość f_2 ustawić na 31.
5. Ostatnia figura: zmienić częstotliwość f_1 na 30.

Uwagi

Moduł DDS używa interfejsów AXI-Stream. W skrócie interfejs działa następująco: sygnał zawierający próbki funkcji sinus i cosinus (`m_axis_data_tdata`) można odczytywać kiedy `m_axis_data_tvalid` ma wartość '1'. Trzeba pamiętać o tym, że mamy 2 kanały – będą one wystawiane naprzemiennie. Po wypełnieniu potoku wewnętrznego sygnał `m_axis_data_tvalid` pozostanie już na stałe włączony i generator osiągnie założoną częstotliwość próbkowania (50 MHz). Jeżeli nie jesteśmy w stanie tak szybko pobierać danych, możemy podać '0' na wejście `aclken`, aby zatrzymać generator. W sygnale `m_axis_data_tdata` znaczące są bity: 26-16 (sinus) i 10-0 (cosinus).

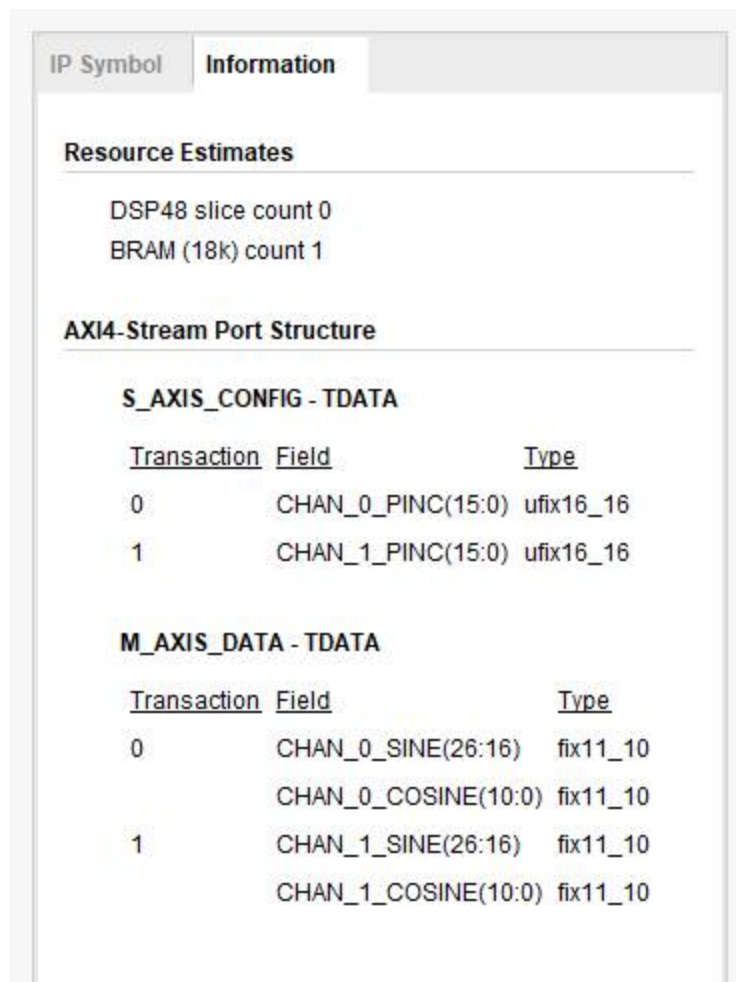
Konfigurację częstotliwości przeprowadzamy także korzystając z interfejsu AXI-Stream. W pierwszej transakcji (włączyć `s_axis_config_tvalid`) należy przesłać dane dotyczące częstotliwości pierwszego kanału. W drugiej i ostatniej transakcji (dodatkowo włączyć `s_axis_config_tlast`) należy przesłać dane dotyczące częstotliwości drugiego kanału. Po transakcji wyłączyć

s_axis_config_tvalid. Pamiętać aby w trakcie transakcji był włączony zegar generatora (sygnał aclken).

W razie wątpliwości co do działania generatora DDS proszę zajrzeć do dokumentacji:

[Dokumentacja kompilatora DDS na stronie Xilinx](#)

Poniżej przedstawiono zakładkę **Information** zawierającą istotne informacje dotyczące ustawień komunikacji AXI-Stream (uwaga – kanały są numerowane od 0):



The screenshot shows the 'Information' tab of an IP Symbol window. It contains two main sections: 'Resource Estimates' and 'AXI4-Stream Port Structure'.

Resource Estimates

- DSP48 slice count 0
- BRAM (18k) count 1

AXI4-Stream Port Structure

S_AXIS_CONFIG - TDATA

Transaction	Field	Type
0	CHAN_0_PINC(15:0)	ufix16_16
1	CHAN_1_PINC(15:0)	ufix16_16

M_AXIS_DATA - TDATA

Transaction	Field	Type
0	CHAN_0_SINE(26:16)	fix11_10
	CHAN_0_COSINE(10:0)	fix11_10
1	CHAN_1_SINE(26:16)	fix11_10
	CHAN_1_COSINE(10:0)	fix11_10

Sygnały event_s_config_tlast_missing i event_s_config_tlast_unexpected można pozostawić jako niepodłączone (open).

Sygnały red_o, green_o i blue_o są czterobitowe – wszystkie ich bity sterujemy równocześnie (same jedynki oznaczają kolor biały – figurę, a same zera oznaczają czarne tło).

Na nagraniu z kamery figura może wyglądać tak, jakby była trochę pokolorowana – można to zignorować. Monitor po wykryciu sygnału włącza się z uśpienia, pełną jasność ekranu osiągnie po około 1 minucie.

Informacje dodatkowe o zasadach działania generatorów DDS:

http://en.wikipedia.org/wiki/Direct_digital_synthesizer

<http://www.ieee.li/pdf/essay/dds.pdf>